



Università
Ca'Foscari
Venezia

Decision Trees

FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

Andrea Torsello

Dip. Sc. Ambientali, Informatica, Statistica
Università Ca'Foscari Venezia

Decision Trees

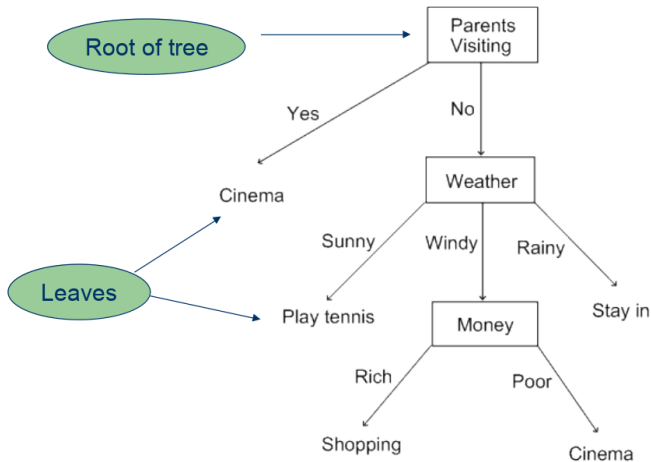
Complex decisions can often be expressed in terms of a series of questions:

What to do this Weekend?

- ▶ If my parents are visiting
 - ▶ We'll go to the cinema
- ▶ If not
 - ▶ Then, if it's sunny I'll play tennis
 - ▶ But if it's windy and I'm rich, I'll go shopping
 - ▶ If it's windy and I'm poor, I'll go to the cinema
 - ▶ If it's rainy, I'll stay in

Decision Trees

These decision can be expressed in terms of a tree



Decision Trees

Decision trees can be written as

- ▶ Horn clauses in first order logic

Read from the root to every tip

- ▶ If this and this and this ... and this, then do this

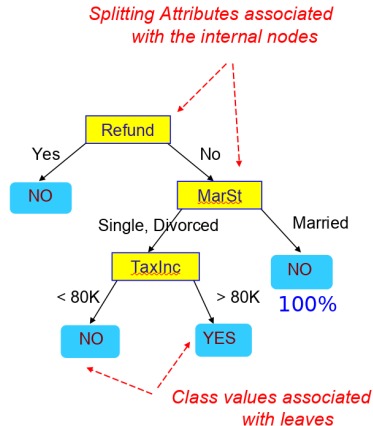
In our example:

- ▶ If `no_parents` and `sunny_day`, then `play_tennis`
- ▶ `no_parents` $\wedge$ `sunny_day` $\rightarrow$ `play_tennis`

Another Example of Decision Tree

<i>Tid</i>	<i>Refund</i>	<i>Marital Status</i>	<i>Taxable Income</i>	<i>Cheat</i>
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

categorical
categorical
continuous
class



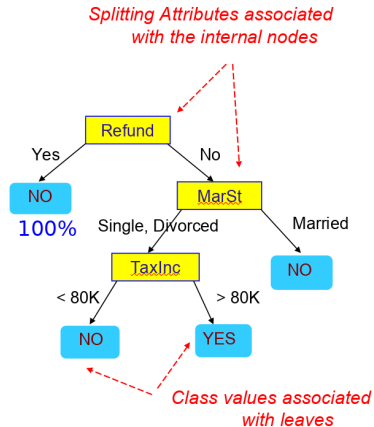
Training Data

Model: Decision Tree

Another Example of Decision Tree

<i>Tid</i>	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

categorical
categorical
continuous
class



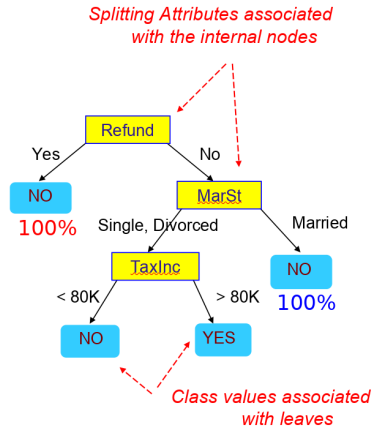
Training Data

Model: Decision Tree

Another Example of Decision Tree

<i>Tid</i>	<i>Refund</i>	<i>Marital Status</i>	<i>Taxable Income</i>	<i>Cheat</i>
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

categorical
categorical
continuous
class



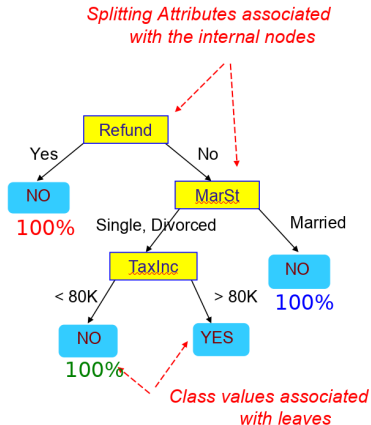
Training Data

Model: Decision Tree

Another Example of Decision Tree

<i>Tid</i>	<i>Refund</i>	<i>Marital Status</i>	<i>Taxable Income</i>	<i>Cheat</i>
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

categorical
categorical
continuous
class



Training Data

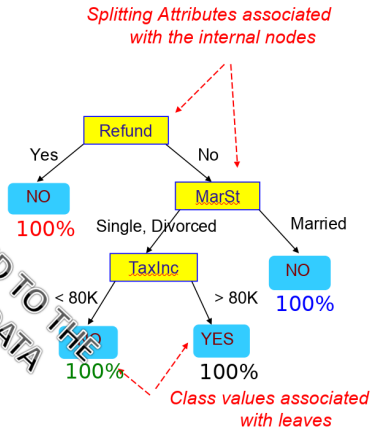
Model: Decision Tree

Another Example of Decision Tree

categorical categorical continuous class

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

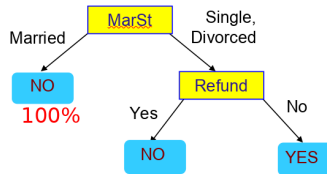
Training Data



Model: Decision Tree

Another Example of Decision Tree

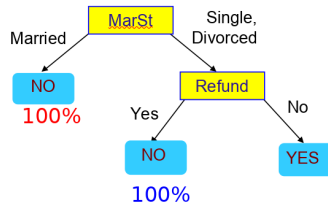
<i>Tid</i>	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



There could be more than one tree that fits the same data!

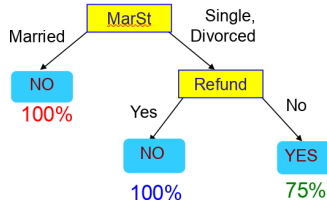
Another Example of Decision Tree

<i>Tid</i>	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



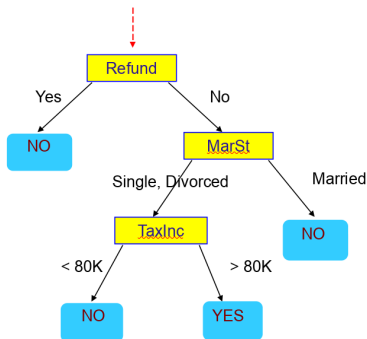
Another Example of Decision Tree

		categorical	categorical	continuous	class
Tid	Refund	Marital Status	Taxable Income	Cheat	
1	Yes	Single	125K	No	
2	No	Married	100K	No	
3	No	Single	70K	No	
4	Yes	Married	120K	No	
5	No	Divorced	95K	Yes	
6	No	Married	60K	No	
7	Yes	Divorced	220K	No	
8	No	Single	85K	Yes	
9	No	Married	75K	No	
10	No	Single	90K	Yes	



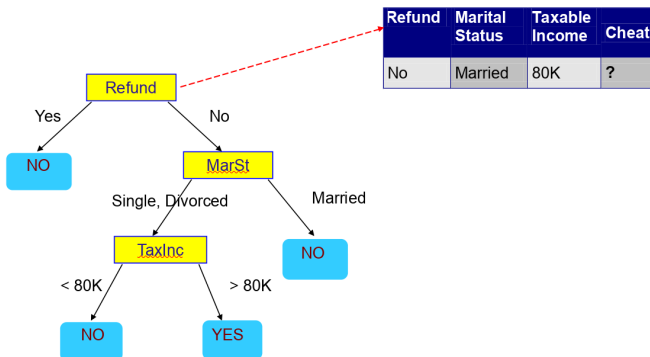
Apply Model to Test Data

Start from the root of tree.

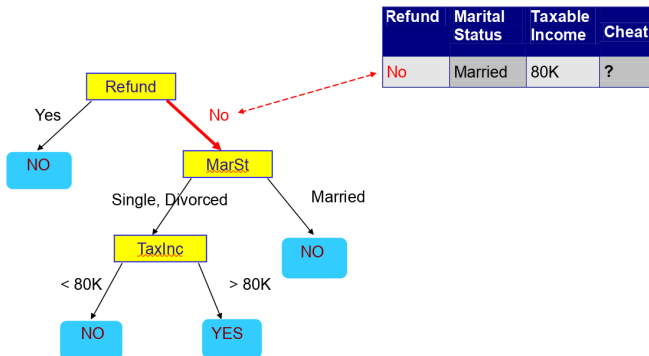


Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

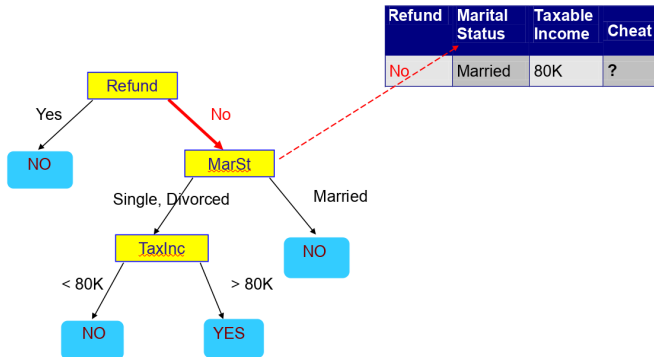
Apply Model to Test Data



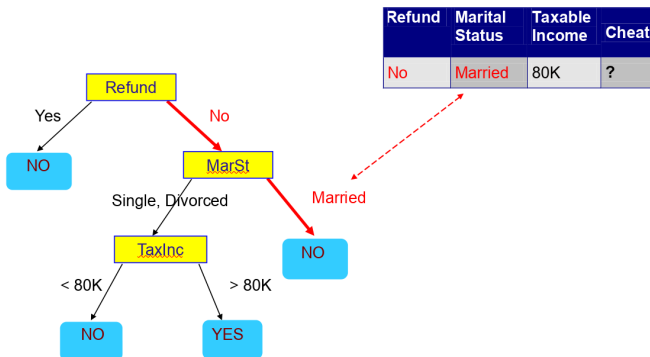
Apply Model to Test Data



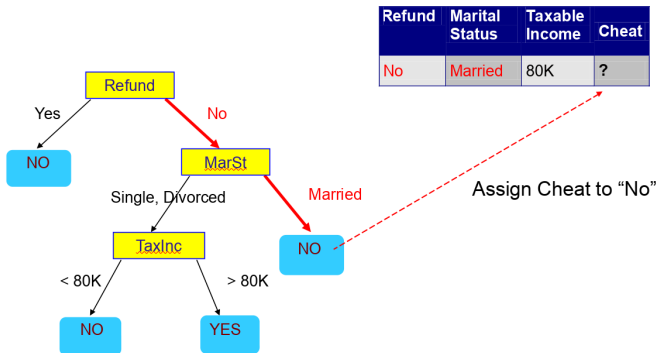
Apply Model to Test Data



Apply Model to Test Data



Apply Model to Test Data



Decision Tree Learning

Decision tree can be seen as rules for performing a categorization

- ▶ E.g., “what kind of weekend will this be?”

Remember that we're learning from examples

- ▶ Not turning thought processes into decision trees

We need examples put into categories

We also need attributes for the examples

- ▶ Attributes describe examples (background knowledge)
- ▶ Each attribute takes only a finite set of values

The ID3 Algorithm

The **major question** in decision tree learning

- ▶ **Which nodes to put in which positions**
- ▶ Including the root node and the leaf nodes

ID3 uses a measure called **Information Gain**

- ▶ Based on a notion of entropy (“Impurity in the data”)
- ▶ Used to choose which node to put in next

Node with the highest information gain is chosen

- ▶ When there are no choices, a leaf node is put on

The randomness of the sampling from the source to the target.

Entropy

For example (binary categorization) given a set of examples S , where

- ▶ p_+ is the proportion of positives
- ▶ p_- is the proportion of negatives

$$\text{Entropy}(S) = -p_+ \log_2(p_+) - p_- \log_2(p_-)$$

For examples in categorizations $c_1, \dots, c_n$

- ▶ p_i is the proportion of examples in c_i

$$\text{Entropy}(S) = - \sum_{i=1}^n p_i \log_2(p_i)$$

Entropy

Each category adds to the whole measure

When p_i is near to 1

- ▶ (Nearly) all the examples are in this category
 - ▶ So it should score low for its bit of the entropy
- ▶ $\log_2(p_i)$ gets closer and closer to 0
 - ▶ And this part dominates the overall calculation
 - ▶ So the overall calculation comes to nearly 0 (which is good)

When p_i is near to 0

- ▶ (Very) few examples are in this category
 - ▶ So it should score low for its bit of the entropy
- $\log_2(p_i)$ gets larger (more negative), but does not dominate
 - ▶ Hence overall calculation comes to nearly 0 (which is good)

Information Gain

Given set of examples S and an attribute A

- ▶ A can take values $v_1, \dots, v_m$
- ▶ Let $S_v = \{\text{examples which take value } v \text{ for attribute } A\}$

Calculate $\text{Gain}(S, A)$

- ▶ Estimates the reduction in entropy we get if we know the value of attribute A for the examples in S

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

Example of Information Gain

Suppose we have a set of examples

$$S = \{s_1, s_2, s_3, s_4\}$$

In a binary categorization

- ▶ With one positive example and three negative examples
- ▶ The positive example is s_1

and one attribute A taking values

$$V = \{v_1, v_2, v_3\}$$

- ▶ s_1 takes value v_2 for A
- ▶ s_2 takes value v_2 for A
- ▶ s_3 takes value v_3 for A
- ▶ s_4 takes value v_1 for A

Example of Information Gain

Recall that

$$\begin{aligned}\text{Entropy}(S) &= -p_+ \log_2(p_+) - p_- \log_2(p_-) \\ \text{Gain}(S, A) &= \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)\end{aligned}$$

from the data, we have

$$p_+ = \frac{1}{4} \text{ and } p_- = \frac{3}{4}$$

Hence

$$\text{Entropy}(S) = -\frac{1}{4} \log_2 \left(\frac{1}{4} \right) - \frac{3}{4} \log_2 \left(\frac{3}{4} \right) = 0.811$$

Example of Information Gain

From the data

$$S_{v_1} = \{s_4\}, S_{v_2} = \{s_1, s_2\}, S_{v_3} = \{s_3\}$$

Substituting, we have

$$\frac{|S_{v_1}|}{|S|} \text{Entropy}(S_{v_1}) = \frac{1}{4} \left(-\frac{0}{1} \log_2(0/1) - \frac{1}{1} \log_2(1/1) \right) = 0$$

Similarly,

$$\frac{|S_{v_2}|}{|S|} \text{Entropy}(S_{v_2}) = \frac{1}{2}$$

$$\frac{|S_{v_3}|}{|S|} \text{Entropy}(S_{v_3}) = 0$$

Finally, we have

$$\text{Gain}(S, A) = 0.811 - (0 + \frac{1}{2} + 0) = 0.311$$

The ID3 Algorithm

Given a set of examples, S

- ▶ Described by a set of attributes A_i
- ▶ Categorised into categories c_j

1. Choose the root node to be attribute A
 - ▶ Such that A scores highest for information gain
 - ▶ Relative to S , i.e., $\text{gain}(S, A)$ is the highest over all attributes
2. For each value v that A can take
 - ▶ Draw a branch and label each with corresponding v

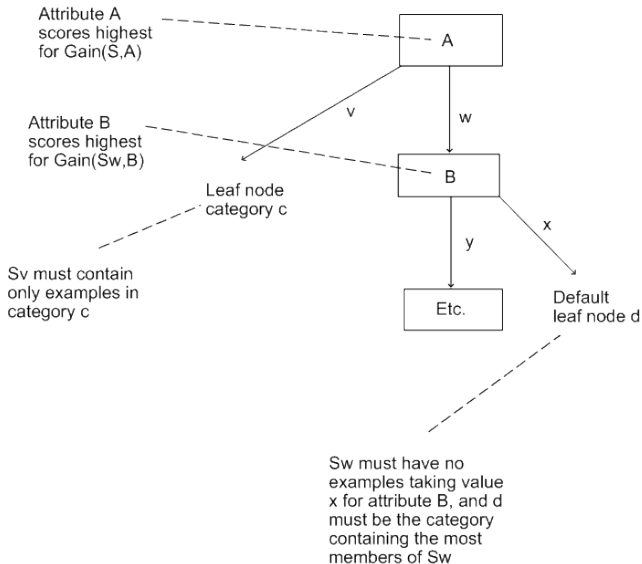
The ID3 Algorithm

For each branch you've just drawn (for value v)

- ▶ If S_v only contains examples in category c That means a single category
 - ▶ Then put that category as a leaf node in the tree
- ▶ If S_v is empty Hypothetically, in the future dataset but not in the current training data
 - ▶ Then find the default category (which contains the most examples from S)
 - ▶ Put this default category as a leaf node in the tree
- ▶ Otherwise
 - ▶ Remove A from attributes which can be put into nodes
 - ▶ Replace S with S_v
 - ▶ Find new attribute A scoring best for $\text{Gain}(S, A)$
 - ▶ Start again at part 2

Make sure you replace S with S_v

The ID3 Algorithm



Example

Weekend	Weather	Parents	Money	Decision (Category)
W1	Sunny	Yes	Rich	Cinema
W2	Sunny	No	Rich	Tennis
W3	Windy	Yes	Rich	Cinema
W4	Rainy	Yes	Poor	Cinema
W5	Rainy	No	Rich	Stay in
W6	Rainy	Yes	Poor	Cinema
W7	Windy	No	Poor	Cinema
W8	Windy	No	Rich	Shopping
W9	Windy	Yes	Rich	Cinema
W10	Sunny	No	Rich	Tennis

Example

$$S = \{W_1, W_2, \dots, W_{10}\}$$

Firstly, we need to calculate:

$$\text{Entropy}(S) = 1.571$$

Next, we need to calculate information gain

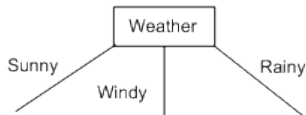
- ▶ For all the attributes we currently have available (which is all of them at the moment)
- ▶ $\text{Gain}(S, \text{weather}) = 0.7$
- ▶ $\text{Gain}(S, \text{parents}) = 0.61$
- ▶ $\text{Gain}(S, \text{money}) = 0.2816$

Hence, the weather is the first attribute to split on

- ▶ Because this gives us the biggest information gain

Example

So, this is the top of our tree:



Now, we look at each branch in turn

- ▶ In particular, we look at the examples with the attribute prescribed by the branch

$$S_{\text{sunny}} = \{W_1, W_2, W_{10}\}$$

- ▶ Categories are cinema, tennis and tennis for W_1 , W_2 , and W_{10}
- ▶ What does the algorithm say?
 - ▶ Set is neither empty, nor a single category
 - ▶ So we have to replace S by S_{sunny} and start again

Example

Weekend	Weather	Parents	Money	Decision
W1	Sunny	Yes	Rich	Cinema
W2	Sunny	No	Rich	Tennis
W10	Sunny	No	Rich	Tennis

Need to choose a new attribute to split on

- ▶ Cannot be weather, of course – we've already had that

Calculate information gain again:

- ▶ $\text{Gain}(S_{\text{sunny}}, \text{parents}) = 0.918$
- ▶ $\text{Gain}(S_{\text{sunny}}, \text{money}) = 0$

Hence we choose to split on parents

Example

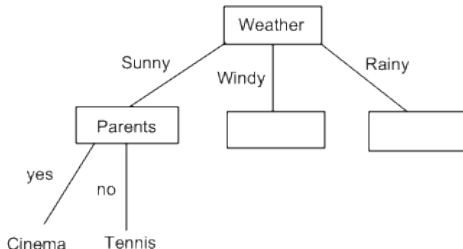
If it's sunny and the parents have turned up

- ▶ Then, looking at the table in previous slide
- ▶ There's only one answer: go to cinema

If it's sunny and the parents haven't turned up

- ▶ Then, again, there's only one answer: play tennis

Hence our decision tree looks like this

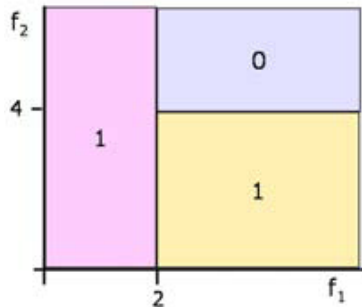
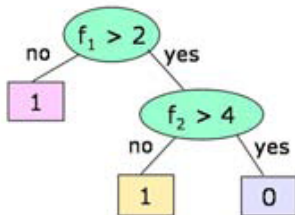


Decision Boundary

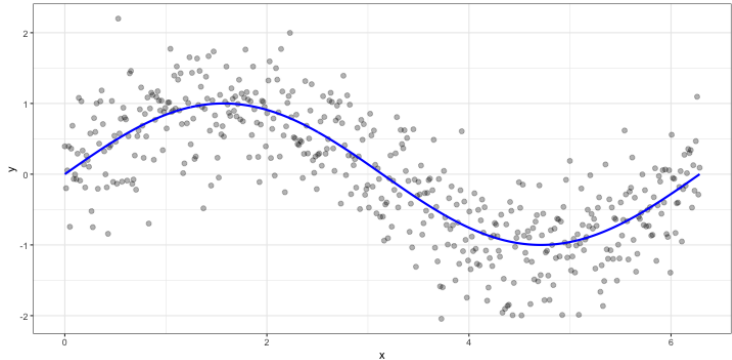
All the attributes used for categorization form an n-dimensional space

Decisions can be of the form $x_i > \text{constant}$

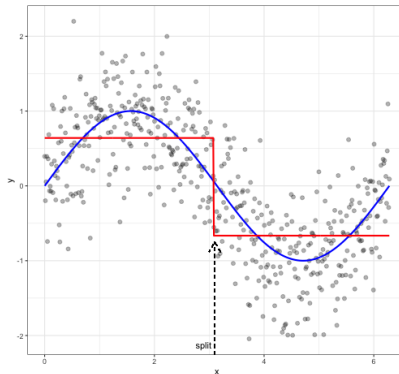
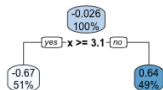
- Divides the space into axis aligned rectangles



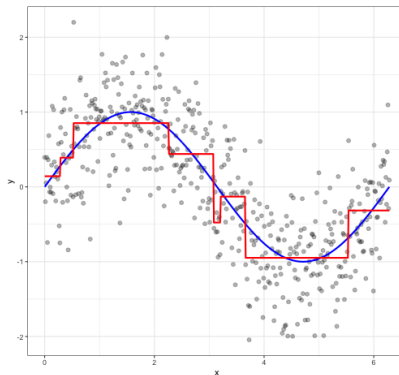
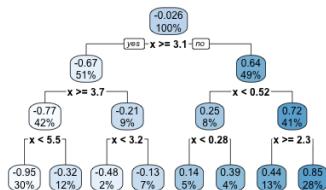
How Deep to Grow a Tree?



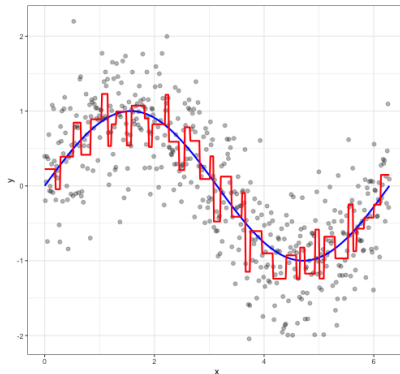
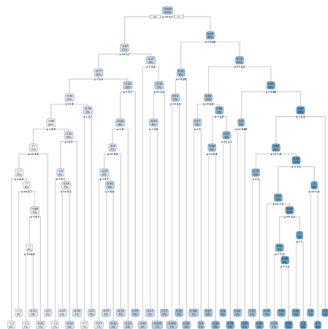
Depth=1 (Decision Stump)



Depth=3



Depth=20 (Complex Tree)



Avoiding Overfitting

Decision trees can be learned to perfectly fit the data given

- ▶ This is probably overfitting
- ▶ The answer is a memorisation, rather than generalisation

Avoidance method 1:

- ▶ Stop growing the tree before it reaches perfection

Avoidance method 2:

- ▶ Grow to perfection, then prune it back afterwards
- ▶ Most useful of two methods in practice

Decision tree algorithms are fairly robust to errors

- ▶ In the actual classifications
- ▶ In the attribute-value pairs
- ▶ In missing information

Strengths & Weaknesses

Strengths

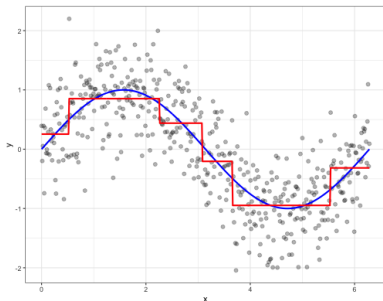
- ▶ Small trees are easy to interpret
- ▶ Trees scale well to large N (fast!!)
- ▶ Can handle data of all types (i.e., requires little, if any, preprocessing)
- ▶ Automatic variable selection
- ▶ Can handle missing data
- ▶ Completely nonparametric

Weaknesses

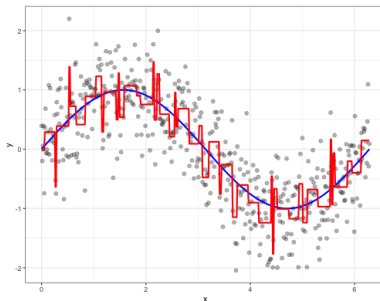
- ▶ Large trees can be difficult to interpret
- ▶ All splits depend on previous splits (i.e. capturing interactions **good**; additive models **bad**)
- ▶ Trees are step functions (i.e., binary splits)
- ▶ Single trees typically have poor predictive accuracy
- ▶ Single trees have high variance (easy to overfit to training data)

The problem with single trees

Single pruned trees are poor predictors

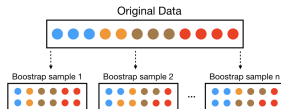


Single deep trees are noisy



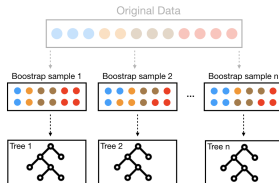
Bootstrap Aggregating (Bagging)

1. Sample records with replacement
(aka "bootstrap" the training data)



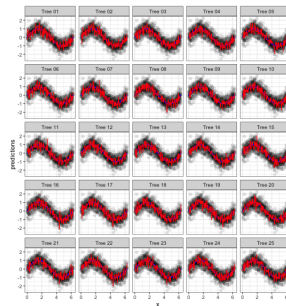
Bootstrap Aggregating (Bagging)

1. Sample records with replacement (aka "bootstrap" the training data)
2. Fit an overgrown tree to each resampled data set



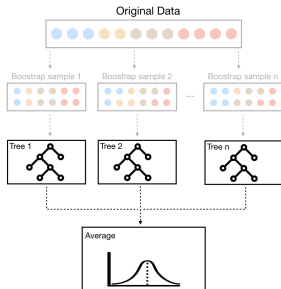
Bootstrap Aggregating (Bagging)

1. Sample records with replacement
(aka "bootstrap" the training data)
2. Fit an overgrown tree to each
resampled data set



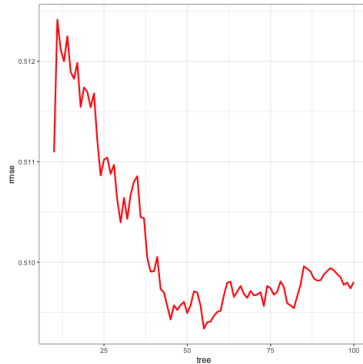
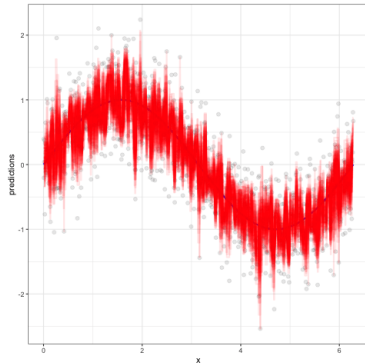
Bootstrap Aggregating (Bagging)

1. Sample records with replacement (aka "bootstrap" the training data)
2. Fit an overgrown tree to each resampled data set
3. Average predictions



Bootstrap Aggregating (Bagging)

As we add more trees... our average prediction error reduces



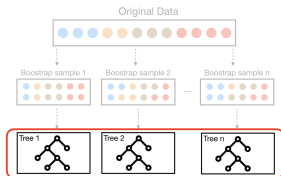
However, a problem remains

- ▶ Bagging results in tree correlation...
- ▶ which prevents bagging from optimally reducing variance of the predictive values

Random Forests

Split-variable randomization

- Follow a similar bagging process but...

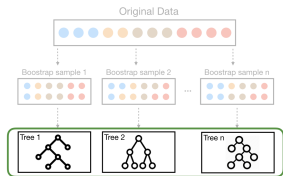


Bagging produces
many correlated trees

Random Forests

Split-variable randomization

- ▶ Follow a similar bagging process but...
- ▶ each time a split is to be performed, the search for the split variable is limited to a random subset of m of the p variables
 - ▶ regression trees: $m = \frac{p}{3}$
 - ▶ classification trees: $m = \sqrt{p}$

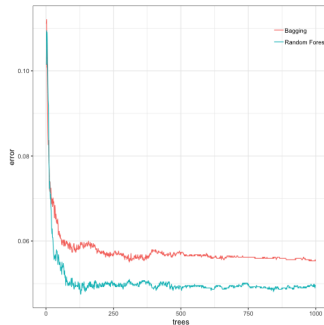


Random Forests
produce many unique
trees

Bagging vs Random Forest

Bagging introduces randomness into the rows of the data

Random forest introduces randomness into the rows and columns of the data



Combined, this provides a more diverse set of trees that almost always lowers our prediction error.

Random Forest Summary

Strenghts

- ▶ Competitive performance
- ▶ Very good “out-of-the box” (very little tuning required)
- ▶ **Built-in validation set** (no need to sacrifice data for validation)
- ▶ Typically does not overfit
- ▶ Robust to outliers
- ▶ Handles missing data (imputation not required)
- ▶ Automatic feature selection
- ▶ Minimal preprocessing

Weaknesses

- ▶ Although accurate, often cannot compete with the accuracy of advanced boosting algorithms
- ▶ Can become slow on large data sets
- ▶ Less interpretable